01. 리눅스의 기본 개념과 프로그램 작성

`

2024. 03. 12
Linux System Programming

- **UNIX**
  - 1969년 AT&T의 벨 연구소에서 어셈블리어로 처음 개발
  - 1971년에 C언어로 재개발
    → 최초의 고급 프로그래밍 언어로 작성한 운영체제로 이식성 높음
  - AT&T의 상용 유닉스와 오픈 소스 버전인 BSD로 나뉘어 발전
  - BSD는 AT&T의 라이선스가 필요 없는 FreeBSD로 발전

- **MS-DOS**
  - IBM이 최초의 IBM PC Model 5150을 개발(1980)
    → 16비트 시스템의 운영체제 외주 개발
  - MicroSoft MS-DOS 1.10 ~ MS-DOS 6.22
  - MicroSoft Windows 95 ~ Windows 2000

- **GNU, GPL & FSF**
  - GNU (GNU's Not Unix) : 리처드 스톨먼(1983)
  - FSF (Free Software Foundation) : 1984
  - GPL : GNU General Public License

- **Linus Torvalds**

  - 포브스(Forbes) (1998년 8월 10일) 인터뷰
    포브스 : 왜 돈도 안 받고 그런 일을 하느냐
    리누스 토르발스 :
    A big part of personal satisfaction is having your work recognized by your peers
    (개인적 만족의 대부분은 자신이 한 일이 동료들의 인정을 받는 것 입니다)

  - Linux(1990) : Linus + Unix
    Linus + Minix

다중 작업 (multi process)

- 선점 가능한(preemptive) 실제 다중 작업을 지원
- 작업은 실행 중인 상태의 프로그램을 의미 (프로세스)

다중 사용자 (multi user)

- 동시에 여러 명의 사용자가 시스템에서 작업하는 것을 허용
- 터미널이나 네트워크 연결을 통해서 동일한 하나의 리눅스 시스템을 사용

`

다중 프로세서 (multi processor)

- 다중 프로세서 구조에서도 실행될 수 있음 (두 개 이상의 CPU를 가진 시스템을 리눅스가 지원)

이식성과 확장성

- 이식성이 높음
- 다양한 언어로 작성된 프로그램을 어렵지 않게 사용할 수 있음

파일 시스템

- 리눅스의 파일 시스템은 유닉스의 것과 같이 트리 구조를 이루고 있음
- 별도로 추가된 물리적인 보조 기억 장치들이나 하드웨어 디바이스들도 파일 형태로 파일 시스템에 연결

권한

- 사용자별로 별도의 권한을 부여
- 하나의 시스템을 여러 명의 사용자가 동시에 사용할 수 있기 때문에 발생할 수 있는 여러 가지 문제를 사전에 방지
- 시스템을 관리하기 위한 관리자와 시스템을 사용하기만 하는 사용자

셸

- 사용자가 시스템을 쉽게 사용할 수 있도록 중간자 역할의 프로그램
- 사용자가 명령어 라인을 입력해서 원하는 작업을 수행할 수 있음

개발 환경

- 리눅스는 새로운 프로그램을 개발하기 위한 환경을 제공
- 프로그래밍 언어용 컴파일러, 프로그램 개발에 필요한 보조적인 유틸리티

- **Linux 명령어**

$ date
$ hostname
$ uname
$ who
$ ls (list)
$ clear
$ pwd (print working directory)
$ mkdir (make directory) ex) mkdir testdir
$ cd (change directory) ex1) cd testdir ex2) cd ..
$ rmdir (remove directory) ex) rmdir testdir
$ ps (processes) ex) ps –ef > a.txt
→ ">" : redirection
$ cat, more, head, tail, wc ex) cat a.txt, more a.txt, …
$ grep (print lines matching a pattern) ex) cat a.txt | grep sys
→ "|" pipe operation
$ cp (copy) ex) cp a.txt b.txt
$ mv (move) ex) mv b.txt c.txt
$ rm (remove) ex) rm c.txt

$ chgrp
$ chmod
$ chown
$ cp
$ df

$ dir
$ dmesg
$ echo
$ grep
$ gunzip

$ gzip
$ kill
$ ps
$ sleep
$ tar

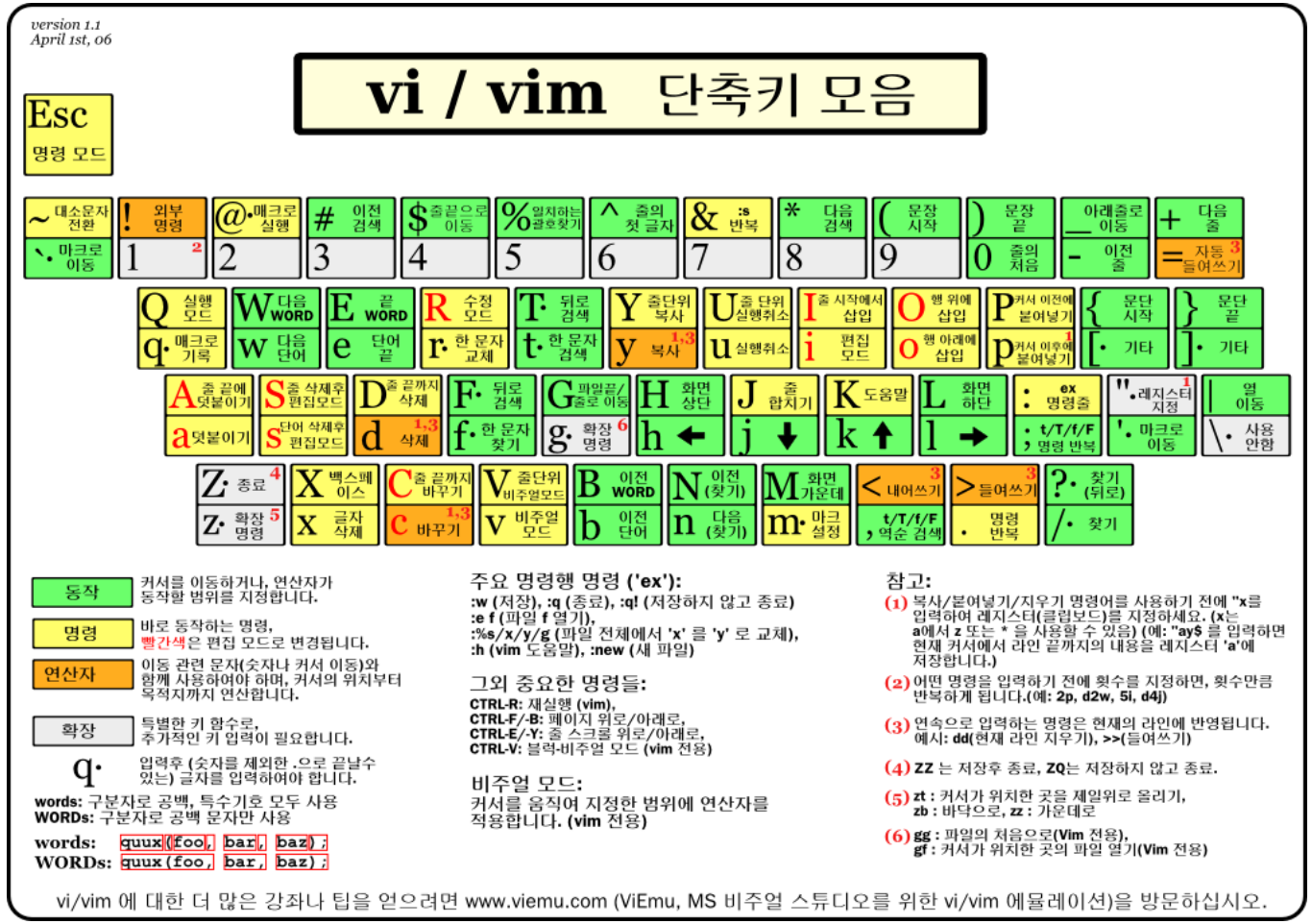
프로그래밍 관련 툴

- 에디터 (editor)
  - 소스 코드를 편집하는 용도로 사용된다.
  - 일반적으로 vi 에디터를 사용한다.

- 컴파일러 (compiler)
  - 소스 코드를 바이너리 코드로 변경한다.
  - 리눅스 시스템에서는 gcc를 사용한다.

- 링커/로더 (linker/loader)
  - 목적(object) 파일들을 연결해서 실행 파일을 만든다.
  - 리눅스 시스템에서는 ld를 사용한다.
  - 대부분 컴파일러가 알아서 실행해준다.

• Vi 명령어



* 명령
- :wq
- :q!
- :h

* 이동
- h,j,k,l
- /xxx
- :n(number)

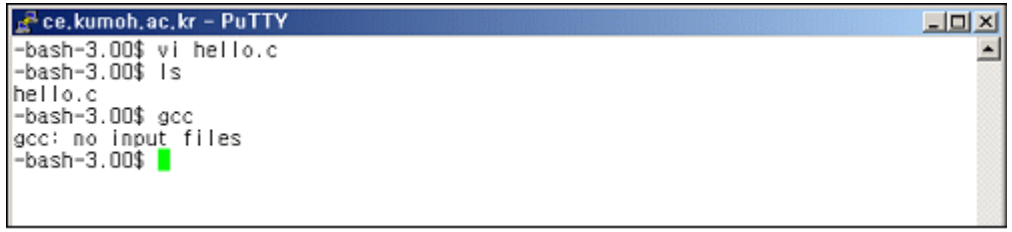
* 편집
- i, I, o, O, a, A
- X, X, dw, dd
- yw, yy, p
- p

* 출처 : https://kldp.org/node/102947

컴파일하기

- gcc (GNU C Compiler)
  - 무료 배포가 원칙인 리눅스에서 일반적으로 사용하는 공개판 C 컴파일러
  - 리눅스를 설치할 때 같이 설치할 수 있다.

- 설치 확인
  - 셸 프롬프트 상태에서 gcc를 실행해본다.
  - gcc: no input files 라는 메시지가 출력되면 설치되어 있다.



make와 Makefile

- make
  - Makefile에서 정한 파일 연관성과 생성 순서에 따라 컴파일(compile)과 링킹(linking) 작업을 수행하여 실행 파일을 작성한다.
- Makefile
  - 하나의 실행 파일을 작성하기 위해서 관련된 소스 코드 파일들을 컴파일하고 링크하는 순서를 정의하고 있는 텍스트 형식의 파일이다.
  - 형식

```
target_list: dependency_list
        command_list
```

  - 예제

```
three: one.c two.c
        gcc -o three one.c two.c
```

- **Makefile 예**

```
ubuntu@ubuntu-desktop:~/ljh$ cat Makefile
out : ex1.o sum.o
	gcc -o out ex1.o sum.o

ex1.o :
	gcc -c ex1.c

ex2.o :
	gcc -c sum.c

clean :
	rm -rf ex1.o sum.o out
```